Name: Vanshika Wadhwani
Class : D20A
Roll no: 35

Experiment: 6

Aim

To create, compile, deploy, and execute smart contracts and blockchain transactions using Solidity programming language and Remix IDE.

Theory

A smart contract is a digital program stored on a blockchain that automatically performs specific actions when predefined conditions are met. It helps reduce the need for intermediaries by executing instructions according to the rules written in the code. Smart contracts are commonly used in decentralized applications (DApps), cryptocurrency transfers, supply chain systems, digital identity management, and decentralized finance.

Solidity is a high-level, statically typed programming language mainly used to develop smart contracts for the Ethereum blockchain and other EVM-compatible networks. It provides features such as variables, functions, constructors, events, and modifiers that help developers create blockchain applications.

Remix IDE is an online development environment that allows users to write, compile, deploy, and test Solidity smart contracts directly through a web browser. It includes a compiler, deployment tools, and a JavaScript virtual machine for testing contracts in a local blockchain environment without requiring additional software installation.

1. Creation of a Smart Contract

The process begins by writing the contract code in Solidity using Remix IDE. A smart contract may include state variables for storing information, a constructor for setting initial values, and functions for accessing or updating the stored data. Solidity also provides visibility keywords such as public, private, internal, and external to control how variables and functions can be accessed.

2. Compilation of the Smart Contract

After writing the code, the Solidity compiler is used to check it for syntax errors and other compilation issues. If the code is valid, the compiler generates bytecode and an Application Binary Interface (ABI). The bytecode is required to deploy the contract on the blockchain, whereas the ABI describes how external applications can communicate with the contract's functions.

3. Deployment of the Smart Contract

Deployment refers to placing the compiled smart contract on a blockchain network. In Remix IDE, deployment can be performed using the JavaScript VM for testing or by connecting a cryptocurrency wallet to a supported blockchain network. The deployment process generates a transaction. Once the transaction succeeds, the smart contract is assigned a unique address that can be used to interact with it.

4. Execution of Smart Contract Transactions

After deployment, the functions of the smart contract can be accessed and executed through Remix IDE. Functions that only read stored information generally do not require a blockchain transaction or gas payment when called locally or through an appropriate read-only call. However, functions that modify blockchain data require a transaction and consume gas.

Gas represents the computational effort required to execute operations on an Ethereum-compatible blockchain. A transaction may contain information such as the sender's address, recipient or contract address, function input data, gas settings, and transferred cryptocurrency value, if applicable.

When the transaction is processed, the blockchain executes the smart contract code and updates its state if the operation succeeds. The transaction receipt contains details such as the execution status and the amount of gas consumed.

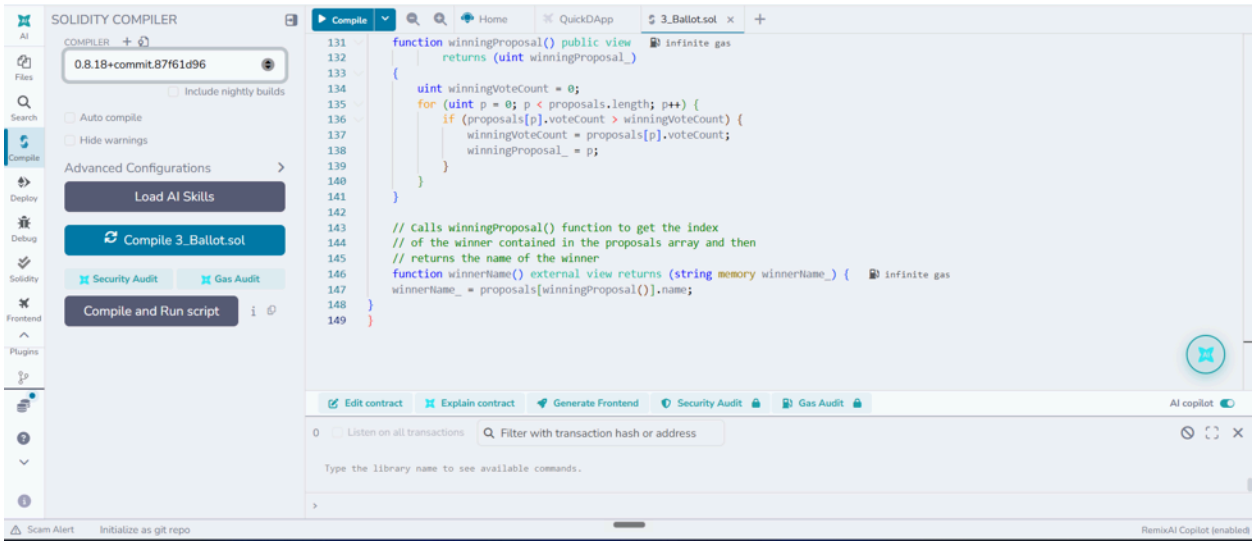
5. Role of Blockchain in Smart Contracts

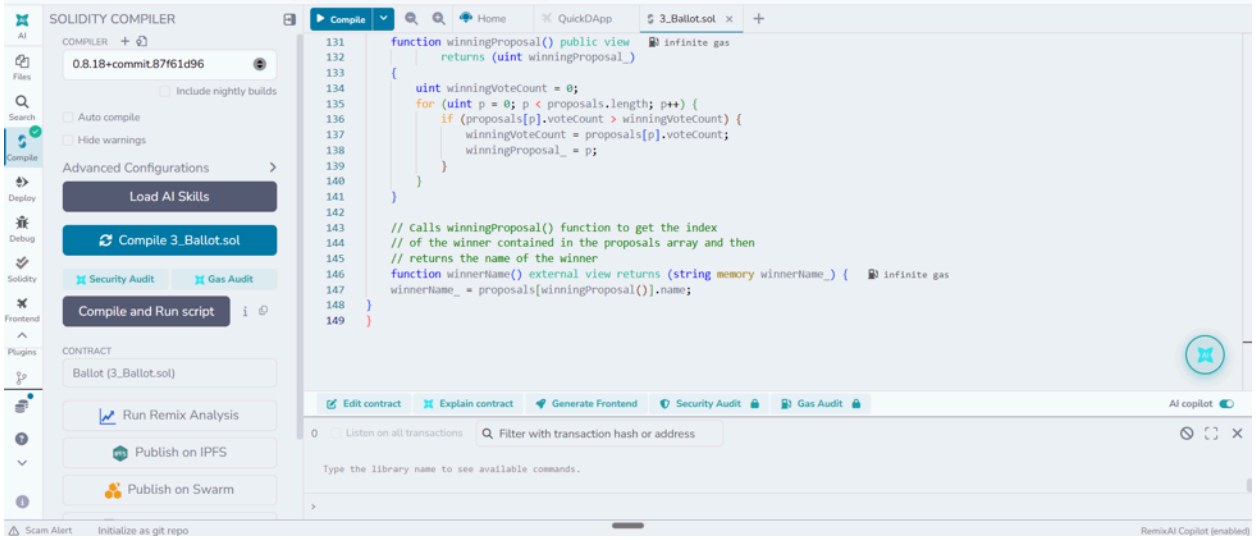
Blockchain technology provides a decentralized and transparent environment for executing smart contracts. It maintains transaction records that can be verified by network participants and makes confirmed records difficult to alter. Smart contracts also help automate processes and reduce dependence on centralized intermediaries.

However, smart contracts must be developed and tested carefully because programming mistakes or security vulnerabilities can lead to incorrect execution and potential financial losses.

Output:


FILE EXPLORER
+ Create
Learneth Solidity Beginner Course
.deps
.states
1. Introduction
10.1 Transactions - Ether and Wei
10.2 Transactions - Gas and Gas Price
10.3 Transactions - Sending Ether
2. Basic Syntax
3. Primitive Data Types
4. Variables
5.1 Functions - Reading and Writing to...
5.2 Functions - View and Pure
5.3 Functions - Modifiers and Construc...
5.4 Functions - Inputs and Outputs
6. Visibility
7.1 Control Flow - If
7.2 Control Flow - Loops
8.1 Data Structures - Arrays
8.2 Data Structures - Mappings
8.3 Data Structures - Structs
8.4 Data Structures - Enums
9. Data Locations
artifacts
contracts
3_Ballot.sol
.prettierrc.json
README.md
Compile
Home
QuickDApp
3_Ballot.sol
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 /// @title Voting with delegation.
5 contract Ballot {
6 // This declares a new complex type which will
7 // be used for variables later.
8 // It will represent a single voter.
9 struct Voter {
10 uint weight; // weight is accumulated by delegation
11 bool voted; // if true, that person already voted
12 address delegate; // person delegated to
13 uint vote; // index of the voted proposal
14 }
15
16 // This is a type for a single proposal.
17 struct Proposal {
18 bytes32 name; // short name (up to 32 bytes)
19 uint voteCount; // number of accumulated votes
20 }
21
22 address public chairperson;
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
AI copilot
Listen on all transactions
Filter with transaction hash or address
Type the library name to see available commands.
Scam Alert
Initialize as git repo
RemixAI Copilot (enabled)


SOLIDITY COMPILER
COMPILER
0.8.18+commit.87f61d96
Include nightly builds
Auto compile
Hide warnings
Advanced Configurations
Load AI Skills
Compile 3_Ballot.sol
Security Audit
Gas Audit
Compile and Run script
Compile
Home
QuickDApp
3_Ballot.sol
131 function winningProposal() public view infinite gas
132 returns (uint winningProposal_)
133 {
134 uint winningVoteCount = 0;
135 for (uint p = 0; p < proposals.length; p++) {
136 if (proposals[p].voteCount > winningVoteCount) {
137 winningVoteCount = proposals[p].voteCount;
138 winningProposal_ = p;
139 }
140 }
141 }
142
143 // Calls winningProposal() function to get the index
144 // of the winner contained in the proposals array and then
145 // returns the name of the winner
146 function winnerName() external view returns (string memory winnerName_) { infinite gas
147 winnerName_ = proposals[winningProposal()].name;
148 }
149 }
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
AI copilot
Listen on all transactions
Filter with transaction hash or address
Type the library name to see available commands.
Scam Alert
Initialize as git repo
RemixAI Copilot (enabled)


SOLIDITY COMPILER
COMPILER
0.8.18+commit.87f61d96
Include nightly builds
Auto compile
Hide warnings
Advanced Configurations
Load AI Skills
Compile 3_Ballot.sol
Security Audit
Gas Audit
Compile and Run script
CONTRACT
Ballot (3_Ballot.sol)
Run Remix Analysis
Publish on IPFS
Publish on Swarm
Compile
Home
QuickDApp
3_Ballot.sol
function winningProposal() public view
returns (uint winningProposal_)
{
uint winningVoteCount = 0;
for (uint p = 0; p < proposals.length; p++) {
if (proposals[p].voteCount > winningVoteCount) {
winningVoteCount = proposals[p].voteCount;
winningProposal_ = p;
}
}
}
// Calls winningProposal() function to get the index
// of the winner contained in the proposals array and then
// returns the name of the winner
function winnerName() external view returns (string memory winnerName_) {
winnerName_ = proposals[winningProposal()].name;
}
}
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
AI copilot
Listen on all transactions
Filter with transaction hash or address
Type the library name to see available commands.
Scam Alert
Initialize as git repo
RemixAI Copilot (enabled)


DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
100.000 ETH
Deploy
Deployed Contracts
Transaction History
EEZ
Deploy
Remix VM osaka
Ballot
3_Ballot.sol
Compiled
proposalNames
string[]
Call data
Params
Auto-Fill with AI Samples
Value
0
wei
Gas limit
auto
0
Compile
Home
QuickDApp
3_Ballot.sol
function winningProposal() public view
returns (uint winningProposal_)
{
uint winningVoteCount = 0;
for (uint p = 0; p < proposals.length; p++) {
if (proposals[p].voteCount > winningVoteCount) {
winningVoteCount = proposals[p].voteCount;
winningProposal_ = p;
}
}
}
// Calls winningProposal() function to get the index
// of the winner contained in the proposals array and then
// returns the name of the winner
function winnerName() external view returns (string memory winnerName_) {
winnerName_ = proposals[winningProposal()].name;
}
}
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
AI copilot
Listen on all transactions
Filter with transaction hash or address
Type the library name to see available commands.
Scam Alert
Initialize as git repo
RemixAI Copilot (enabled)


DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Deploy
Deployed Contracts 1
Transaction History 1
Ballot
0xD7A...F771B
Remix VM
2s ago
Functions
Search functions...
delegate (address)
to (address)
Transact
Calldata
Params
Auto-Fill with AI Samples
giveRightToVote (address)
voter (address)
Transact
Compile
Home
QuickDApp
3_Ballot.sol
function winningProposal() public view
returns (uint winningProposal_)
{
uint winningVoteCount = 0;
for (uint p = 0; p < proposals.length; p++) {
if (proposals[p].voteCount > winningVoteCount) {
winningVoteCount = proposals[p].voteCount;
winningProposal_ = p;
}
}
}
// Calls winningProposal() function to get the index
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
AI copilot
Listen on all transactions
Filter with transaction hash or address
- Select a Javascript file in the file explorer and then run `remix.execute()` or `remix.exeCurrent()` in the command line interface
- Right-click on a JavaScript file in the file explorer and then click `Run`
The following libraries are accessible:
ethers.js
Type the library name to see available commands.
Error encoding arguments: TypeError: expected array value (argument="", value="", code=INVALID_ARGUMENT, version=6.14.0)
[vm] from: 0x5B3...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...00000 logs: 0 hash: 0x0c0...0094f
Debug



Conclusion:

Thus, the smart contract was successfully created, compiled, and deployed using Solidity and Remix IDE. The contract functions were executed to perform transactions and observe changes in blockchain state. This experiment helped in understanding the complete smart contract development process, including compilation, deployment, transaction execution, gas consumption, and the role of blockchain in providing transparency, security, and reliability.